F1 Predictor V4

Clean-restart plan. Phase by phase, kept deliberately simple.

Written October 20, 2026. Supersedes the earlier V4 research specification (F1_Predictor_V4_Complete_Plan.pdf) and the earlier V4 notebook.

> **The one goal.** After qualifying and before the race starts, predict the probability that each driver finishes on the podium (top 3), and show the predicted top three. **Success** means beating a plain grid-slot baseline on out-of-sample Brier score, on races that were not used for any tuning. Accuracy is never used as the headline number.

1. Why a clean restart

What the V3 notebook got wrong (read directly from the code):

- **Accuracy is misleading here.** Only 15% of driver-race rows are podiums, so a model that never predicts a podium already scores about 85%. V3 reported 93% on 180 rows (9 races, 2025 R11 to R19).
- **Constructor features leak the future.** They were computed on rows sorted by driver, not by date, so a later-sorted teammate can see the other driver's whole-season points, including races that had not happened yet. `constructor_last3_avg_points` was V3's second most important feature.
- **The 'circuit' features are not per-circuit.** They group by driver only, so they are career totals. `circuit_avg_position_change` also averages the current race's own result.
- **Missing values were filled with 0**, which tells the model that a missing time is a real number.
- **The final model was picked on the same small test set** it was reported on, and the shipped app only handled the top 10 qualifiers at a limited set of circuits.

What went wrong in the first V4 attempt:

- Twenty cells of data plumbing and evaluation setup before a single predictive model existed.
- Code was stored as long strings inside notebook cells, which made it hard to read and change.
- The scope kept growing (eight experiments E0 to E7, six-week milestones, joint ranking models), and the plan changed from cell to cell.

What is worth keeping from that attempt (knowledge only, to be re-verified in the new build):

- A smoothed grid-slot lookup is a very strong baseline. On 55 development races (2024 R1 to 2026 R7) it scored a Brier of about **0.068**, against about 0.100 for a historical per-driver podium rate. **This is the bar to beat.**
- On the 8 races V3 could be scored on, V3 as shipped scored a Brier of about 0.224, against about 0.137 for the grid-slot lookup. The sample is small and partly in-sample, so treat it as a warning, not a verdict.
- Data quirks found: Jolpica session dates were wrong for Las Vegas 2023 and 2024; Miami 2025 has no Q1 times at all; a few entries have no qualifying row; pit-lane starts appear as grid position 0; disqualified drivers need to be detected from the status text. FastF1 session data returned 403 errors in the earlier environment, so do not make it a dependency.

2. The rules (the contract)

Topic	Rule
Row and target	One row per driver per race. y = 1 if the official final classification position is 1, 2 or 3, otherwise 0. DNF, DNS and disqualified drivers are rows with y = 0.
Cutoff	Only information available after the qualifying session that sets the Grand Prix grid and before the race. Features for a race may only use earlier races, sorted by date.
Leak test	Features for race r must be identical after deleting every race from r onward. This test runs automatically and must pass before any model is trained.
Missing data	Stays NaN. Never fillna(0). Missing qualifying times get an explicit missing flag.
Primary metric	Brier score (lower is better). Also report log loss, a calibration table, and the per-race sum of probabilities (should be near 3).

Topic	Rule
Uncertainty	Confidence intervals by resampling whole races, not individual drivers, because drivers in one race are not independent.
Baselines	E0: historical podium rate per driver. E1: smoothed grid-slot podium rate (slots 1 to 10 separately, 11 and below pooled, pit-lane starts separate). Both are defined before any results are seen and are never tuned.
Data split	Dev races: expanding-window backtest (train on earlier races, predict the next one). Holdout: the last 10 completed races, opened exactly once in Phase 5. Default holdout is 2026 R8 to R17; confirm in Phase 2.
Code layout	One notebook plus four small files: data.py, features.py, models.py, evalkit.py. Notebook cells only call functions and print results.
Reproducibility	Fixed random seed, library versions logged, data saved as one versioned parquet snapshot with a hash.
Scope control	No new ideas mid-phase. They go on the 'Later' list at the end. A phase is finished only when its 'Done when' line is true.

3. The phases

Same linear flow as V3 (collect, merge, features, train, select, predict), with the evaluation harness added before any modelling.

PHASE 0 | Setup	
Goal	Use existing Python; no virtual environment or Python version downgrade.
Steps	• Create the project folder with a notebook and the four .py files (empty stubs). • Install dependencies compatible with existing Python; pin resolved versions in requirements.txt (pandas, pyarrow, requests, scikit-learn, catboost, lightgbm or xgboost, matplotlib). • Create a config in one place: seasons 2022 to 2026, seed, paths, holdout size.
Output	Folder structure, requirements.txt, config.
Done when	A fresh kernel using the existing Python imports everything without errors.

PHASE 1 | Data (V3 steps 1 to 3)	
Goal	One clean table with one row per driver per race, for completed races 2022 to 2026.
Steps	• Pull races, results and qualifying from Jolpica (Ergast-compatible). Cache the raw JSON so reruns never re-hit the API; respect rate limits. • Use stable driver and constructor IDs (not names or three-letter codes). • Merge into one `entries` table: season, round, date, driver, constructor, grid slot, Q1/Q2/Q3 seconds, final position, status, y_podium. • Record the qualifying session time per race so the cutoff is explicit. Check dates against the known Las Vegas 2023 and 2024 errors. • Run sanity checks and print a short coverage report.
Output	One parquet snapshot plus a short data_notes.md listing every known gap and how it was handled.
Done when	Every race has exactly 3 podium rows; about 20 drivers per race; missing qualifying rows, pit-lane starts and disqualifications are counted and handled by a written rule.

PHASE 2 | Evaluation harness and baselines	
Goal	A single function that scores any model fairly. This becomes the bar every later model must beat.
Steps	• Freeze the dev/holdout split in config. Nothing reads holdout labels yet. • Write metrics in evalkit.py: Brier, log loss, calibration table, per-race probability sum, race-level bootstrap intervals. Unit-test them on tiny synthetic data. • Write `backtest(model, features)`: expanding window, retrain before each race (or each block of races), save every prediction with a timestamp. • Run E0 and E1 through it. • Optional: score V3 as shipped on the races it can cover, clearly labelled partial coverage.
Output	A results table for E0 and E1 on dev races, with confidence intervals.
Done when	Metric tests pass, E1 is reproduced (expect a dev Brier near 0.068), and a plot shows the holdout was never accessed.

PHASE 3 | Features (V3 steps 4 to 9, smaller and leak-proof)	
Goal	A small, honest feature set where each group has to earn its place.
Steps	• Write one function `build_features(race)` that sees only earlier races. Sort by date, never by driver. • **A. Qualifying:** grid slot, gap to pole in percent, made Q3, gap to teammate, pit-lane start flag. • **B. Driver form:** average finish over the last 5 races, podium rate over the last 10, DNF rate. • **C. Team strength:** the team's average finish and podium rate over its last 5 races, computed from earlier races only. • **D. Circuit history:** a driver's podium rate at that circuit, shrunk toward their overall rate when data is thin. • Run the leak test. Then add one group at a time to E1 and keep it only if paired dev Brier improves with the interval mostly below zero.
Output	features.py, a leak-test pass, and an ablation table (group added vs dev Brier change).
Done when	The leak test passes and the kept feature groups are listed with evidence. Target is roughly 15 to 20 features in total.

PHASE 4 | Models (V3 steps 10 to 12)	
Goal	Find out which learner extracts the signal, using the harness only.
Steps	• Logistic regression on the kept features (simple reference). • CatBoost (V3's winner) with NaN left as NaN and a small tuning grid, tuned on dev races only. • Optionally one more booster (LightGBM or XGBoost). • Calibration (Platt or isotonic), fitted using earlier races only. • One test: rescale each race's probabilities so they sum to 3, and see whether Brier improves.
Output	A comparison table of every model against E1 on the same dev races, with intervals.
Done when	One model is chosen by dev Brier. If nothing clearly beats E1 on dev, say so and do not proceed as if it did.

PHASE 5 | Final exam on the holdout (one shot)	
Goal	Honest out-of-sample evidence.
Steps	• Unlock the holdout once. Run the chosen model, E1 and (if available) V3 as shipped on the same races. • Report Brier, log loss, calibration and race-level probability sums with intervals. • Write a one-page results note: what worked, what did not, where it fails (by season, circuit, and wet or chaotic races).
Output	results.md and final plots.
Done when	Decision recorded. **Rule:** the model must beat E1 on holdout Brier with the interval clear of zero. Ten races is a small sample, so the interval will be wide. If it does not beat E1, the honest finding is that a grid-slot lookup is hard to beat, and the write-up says so.

PHASE 6 | Forward test	
Goal	Prove it on races that have not happened yet.
Steps	• After qualifying, freeze the model and save timestamped predictions before the race starts. Score them afterwards. • Repeat for each remaining 2026 round.
Output	A prediction log with timestamps and a running score against E1.
Done when	At least a few real races are logged and scored. Note: the next event, Austin (October 23 to 25), is probably too soon for a finished model. A reasonable use of Austin is to log E1 predictions as a dry run of the process.

PHASE 7 | predict_race() (V3 step 13)	
Goal	A simple way to get the prediction for an upcoming race.
Steps	• One function: input season and round, output the full driver table with podium probabilities and the predicted top three. • Handle the whole field (V3 only handled the top 10 qualifiers).
Output	predict_race() and a saved example output.
Done when	Build this only if Phase 5 shows a real lift over E1.

4. Later list (explicitly out of scope for now)

- Joint ranking or simulation models such as Plackett-Luce, only if Phase 4 shows clear room for improvement.
- Weather features (needs verified historical snapshots), FastF1 session or telemetry features.
- Sprint-weekend specifics. Version 1 treats the Grand Prix qualifying session as the cutoff and ignores sprint results.
- Dashboard or app rewrite, live inference, deployment, MLflow tracking.

5. Glossary

Term	Meaning
Brier score	Mean squared difference between predicted probability and the 0/1 outcome. Lower is better. Predicting 15% for everyone scores about 0.13.
Calibration	If the model says 70% for many rows, about 70% of them should be podiums.
Expanding-window backtest	Train on all earlier races, predict the next race, then add that race to the training data and repeat.
Dev / holdout	Dev races are used freely for decisions. Holdout races are looked at once, at the end.

Term	Meaning
Per-race probability sum	The podium probabilities of all drivers in a race should add up to about 3, because exactly three drivers finish on the podium.